AWK

Part—2

In the previous article (http://bit.ly/CjKTz), we had covered the basics of Awk. We discovered how an Awk program divides the input file(s) into records and fields, what patterns and actions are, built-in variables, control statements and the functions provided by Awk. In this article, we will discuss advanced Awk functionality, including string- and time-manipulation functions, associative arrays and user-defined functions. These can be very useful for a systems administrator to quickly summarise and analyse the information available in various log files.

To begin with, let's take a look at how some of the advanced string-manipulation functions are used. One of the most useful functions available in Awk is the *split* function. It takes three parameters: a source string, an array to store the split elements (starting from index 1) and the optional field separator *fs*. The *fs* can also be a regular-expression. The *split* function is very useful whenever there is a need for more than one delimiter; for example, if the Squid log has entries like the following:

```
1300063856.476    199 127.0.0.1 TCP_MISS/204 396 GET http://www.
google.co.in/csi? - DIRECT/74.125.71.104 text/html
1300063865.415  28710 127.0.0.1 TCP_MISS/200 18303 CONNECT www.
google.com:443 - DIRECT/74.125.71.104 -
```

To fetch the FQDN or IP for a URL from each log entry, using the default FS of " ", check for the occurrence of the pattern *GET* in the 6th field—and if found, split the 7th field into an array *url*, using the delimiter as "/", and print the 3rd index of *url*:

```
awk '$6~/GET/{split($7,url,"/"); print url[3]}' /var/log/squid/
access.log
www.gmail.com
mail.google.com
google.co.in
www.google.co.in
```

Another very important string manipulation function